InferenceOS

Adaptive model serving gateway and inference control plane

One OpenAI-compatible endpoint that routes requests to the right local model server with predictable latency, fairness, and failure recovery.

AT A GLANCE	PROJECT PLAN
Primary skills	Rust, async systems, API design, scheduling, AI infrastructure
Cost category	$0 subscription; GPU-scale benchmark is hardware-dependent
Build horizon	10-14 weeks focused
Hardware profile	CPU mode: 16 GB preferred; realistic high-throughput GPU optional
Best resume target	AI Infrastructure / Backend / Systems Performance

01 The complete idea

A SaaS engineering team uses several models for summarization, extraction, and coding assistance. Traffic is spiky; one model server gets saturated, another is idle, and long requests block short ones. InferenceOS is a self-hosted gateway that accepts a standard chat-completions request, estimates demand, checks model capability and tenant budget, and chooses a healthy backend. The user sees one API even when the platform runs a mixture of local llama.cpp servers, Ollama instances, or compatible vLLM workers.

This is not a model-training project and not another chat UI. The engineering contribution is the control plane: measured queueing policies, cancellation, streaming propagation, resource-aware routing, and failover without duplicate charging. It should expose why each scheduling decision was made.

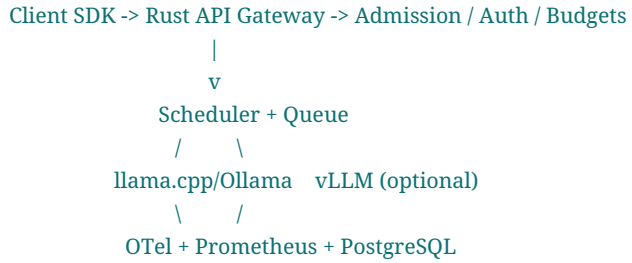
02 Target users and measurable outcomes

- Platform engineer: deploys new backends and sets per-tenant fairness weights.
- Application developer: streams responses without needing to know which model serves them.
- SRE: traces errors, queue delay, backpressure, capacity, and service-level objectives.

03 Concrete product workflow

1. Register two local OpenAI-compatible model endpoints with distinct max concurrency and capabilities.
2. Client sends a chat request with model class, latency tier, max tokens, and an idempotency key.
3. Gateway authenticates the client, validates tokens/budget, and admits request into a bounded queue.
4. Scheduler estimates cost and queue wait, picks a healthy worker, and forwards a streaming or non-streaming request.
5. On worker timeout or failure, retry only when replay is safe; return structured error otherwise.
6. Metrics dashboard displays latency, tokens/sec, queue time, budget usage, and routing reasons.

04 Architectural blueprint



Component	Responsibilities and engineering detail
API edge	Axum HTTP/SSE; request validation, auth, cancellation forwarding, idempotency and correlation IDs.
Admission controller	Per-tenant rate limits and token budgets; bounded queues, 429/503 overload responses, priority starvation limits.
Router/scheduler	Baseline round-robin; least-inflight and estimated-queue-delay algorithms; configurable health probes.
Backend adapter	Normalize OpenAI-compatible streaming semantics, finish reasons, usage accounting, and timeouts.
Telemetry plane	OpenTelemetry traces; Prometheus histogram for TTFT, total latency, error classes and cost estimates.
Control API	Register backends, mark draining, set routing weights, inspect service state; guarded by admin auth.

05 Features: MVP, v1, and advanced

Release	Features and clear boundary
MVP - usable	single Rust gateway; two local backends; round-robin routing; SSE streaming; health checks; Prometheus metrics
v1 - reliable	load-aware router; bounded queue; token budgets; auth/tenant isolation; retries with safety rules; circuit breaker; dashboard
Advanced - optional	dynamic model capability ranking; canary rollout; batch-aware scheduling; prefix-cache locality hints; autoscaling simulation

06 Free stack: exact choices and why

Layer	Zero-subscription solution	Purpose / fallback
API	Rust + Tokio + Axum	High-concurrency gateway; HTTP streams + cancellation.
Model servers	llama.cpp server / Ollama	Local small models; no external paid token API.
GPU option	vLLM self-hosted	Use only if compatible local GPU; CPU alternatives are valid.
Database	PostgreSQL	Persistent tenants, backend configs, budgets, audit records.
Observability	OpenTelemetry + Prometheus + Grafana	All self-hosted locally; no hosted monitoring fee.
Packaging	Podman Compose / Docker Compose	Run gateway + database + two fake/real

Layer	Zero-subscription solution	Purpose / fallback
		model backends.
Testing	k6 / Locust + Criterion	Synthetic OpenAI-compatible backend and load tests.

07 Key contracts and persistence model

Proposed external interfaces (adapt route names as implementation details change):

- POST /v1/chat/completions: ChatCompletionRequest -> SSE tokens or JSON response.
- GET /admin/backends: backend_id, status, queue_depth, concurrency_limit, capabilities.
- POST /admin/backends: register endpoint URL and supported model IDs; must restrict destination hosts (SSRF defense).
- GET /metrics and GET /healthz; GET /readyz only returns ready when configured dependencies are available.

Minimum durable entities or records:

- backend(id, url, model_ids, concurrency_limit, weight, state, updated_at)
- tenant(id, token_budget, max_inflight, allowed_models, policy_version)
- request(id, tenant_id, idempotency_key, selected_backend, state, latency_ms, tokens_estimated, tokens_used)
- route_decision(request_id, candidates, exclusion_reasons, policy, selected_at)

08 Local-first deployment with $0 subscriptions

1. Install Rust, Podman or Docker (personal/educational terms), PostgreSQL container, Prometheus and Grafana locally.
2. Run two deterministic stub-model servers first: each emits fake tokens with configurable wait/fail/drop rates.
3. Bring up gateway with compose profile `cpu-demo`; load config from .env.example with fake secrets.
4. Swap one stub for a small quantized GGUF model via llama.cpp or Ollama and verify streaming end to end.
5. Run repeatable workload scripts; archive anonymized aggregate metrics and benchmark environment info.
6. Optionally use a compatible existing GPU and self-host vLLM; never make GPU a prerequisite for the core project.

No OpenAI/Anthropic subscription needed. CPU-based inference and simulated backends prove correctness and scheduling behavior; they do NOT substantiate claims of enterprise GPU throughput. GPU memory, electricity, and 24/7 hosting are separate costs.

09 Step-by-step engineering roadmap

Phase	Implementation tasks	Deliverable / exit condition
1 - Spec	Define API/streaming protocol, failure semantics, routing invariants, workload generator.	Contract tests and ADR for retry/idempotency policy.
2 - Edge	Implement Axum endpoint, SSE piping, auth middleware, graceful shutdown.	End-to-end streaming and cancellation tests pass.
3 - Routing	Implement registry, round robin, least-inflight, bounded queues.	Deterministic routing unit tests + overload handling.
4 - Persistence	Store tenant limits, configuration	Schema migrations and restart tests.

Phase	Implementation tasks	Deliverable / exit condition
	changes, request summaries.	
5 - Resilience	Add timeouts, health probes, circuit breaking, safe retry contracts.	Fault injection suite demonstrates bounded failure impact.
6 - Bench	Build load generator with prompt length distributions; compare schedulers.	p50/p95/p99 charts with raw reproducible data.
7 - Ops	Grafana dashboards, alerts, compose/kind manifests, CI, runbooks.	One-command run + clean teardown + documented recovery.

10 Production-grade verification and failure testing

- Streaming: chunk ordering, client disconnect, unexpected worker EOF, SSE parse errors, late usage frame.
- Resilience: worker crash mid-stream, transient network partition, 500 storms, warm-to-cold backend transitions.
- Fairness: ten tenants with unequal loads; assert bounded starvation and budget enforcement.
- Security: token auth, SSRF protection on admin backend registration, output redaction, non-leaking request logs.
- Correctness: tokens billed only once, idempotency key under retries, no routing to draining backend.

11 Benchmark plan: what to measure and prove

Metric	Baseline and measurement method	Success criterion (target, not result)
TTFT p95	Same request mix, identical two backends; compare round-robin vs queue-aware.	Target: demonstrable improvement; report actual %
Throughput	Complete requests/min and tokens/sec at 1, 5, 20 concurrent clients.	Target: stable plateau before overload, not an invented TPS
Errors under failure	Kill one worker at 50% load; count failed and correctly surfaced requests.	Target: no stuck requests; defined retry/fail outcome
Fairness	Share of served work and p95 queue wait by tenant.	Target: no unbounded tenant starvation

Metrics above are proposed engineering targets, not measured results. Publish run parameters, machine specs, raw logs, warmup method, and repeated trials. Avoid comparisons across unlike hardware.

12 Security, compliance, and safety decisions

- Never log prompt content by default; use IDs, redacted metadata, and sampling.
- Use strong admin authentication; allowlist model endpoints to mitigate server-side request forgery.
- Separate soft token estimates from actual model-reported usage, clearly documenting inaccuracies.
- Pin dependencies and scan images; keep all model licenses and weight redistribution rules visible.

13 Real risks, trade-offs, and what NOT to build

- True cross-backend retry during streamed output may produce duplicate or inconsistent output; by default never auto-retry after streaming has started.
- Different model tokenizers make budget estimates imperfect: choose an explicit estimated-vs-actual accounting policy.
- Large-model serving will not be free on hardware you do not own; load simulation is not equivalent to GPU throughput.

- Avoid spending weeks training models or building GPU kernel optimizations; the differentiated work is the gateway.

14 Showcase: recruiter-ready live demo

1. Start two fake backends, one slow and one fast; show baseline load distribution.
2. Send mixed workloads and compare p95 latency and per-backend queue utilization.
3. Terminate one worker while requests are active; show precise failure semantics and recovery.
4. Stream from a real locally hosted small model through the same endpoint.
5. Open traces and Grafana; reproduce plots directly from repository scripts.

15 GitHub deliverables and resume story

- Repository should include: README, architecture/design decisions, MIT/Apache-compatible dependency-license review, config examples, .env.example (no secrets), reproducible compose files, CI, tests, benchmarks, dashboards/screenshots, and a CHANGELOG.
- Ship a versioned release tag and an honest limitations section that differentiates local tests from actual multi-host production.
- Add a 2-4 minute screen recording walking through a real failure case and the measured improvement, not only the happy path.

Resume headline (only after building): InferenceOS - adaptive Rust model gateway and inference scheduler.

Resume bullet template (replace brackets with reproducible measurements): Built a Rust inference gateway with token streaming, bounded admission, tenant budgets, and fault-aware routing; improved [p95 TTFT] by [measured %] versus [baseline] on [hardware/workload].

16 Source documentation and technical references

- llama.cpp — https://github.com/ggml-org/llama.cpp
- Ollama documentation — https://docs.ollama.com/
- vLLM supported backends — https://docs.vllm.ai/en/latest/getting_started/installation/
- Rust Axum — https://docs.rs/axum/latest/axum/
- Prometheus documentation — https://prometheus.io/docs/introduction/overview/
- GitHub Actions - billing and usage — https://docs.github.com/en/actions/concepts/billing-and-usage
- Podman - open-source containers — https://docs.podman.io/en/latest/Introduction.html
- Docker Desktop - licensing for personal/education use — https://docs.docker.com/subscription-billing/desktop-license
- kind - local Kubernetes clusters — https://kind.sigs.k8s.io/
- GitHub Pages - hosting limitations — https://docs.github.com/en/pages/getting-started-with-github-pages/github-pages-limits

17 What $0 actually means

Free: coding, source control, local containers, local databases, self-hosted OSS components, realistic test data, reproducible demos, and standard public-repository GitHub Actions runners within current terms/limits. Not promised: electricity, hardware purchases, unrestricted cloud GPU capacity, public always-on servers, custom domains, or zero operational cost for real customers.

Use local/ephemeral deployments and recorded demos for the resume. For public access, a static documentation/demo site can be hosted separately from the backend; GitHub Pages is not a general-purpose server or a commercial SaaS host. Check tool licensing, provider quotas, and model/data licenses before public release.